



Resumen Parcial 2

Unidad 2: Gestión Lean-Agile de Productos de Software

Principios Ágiles

Los principios ágiles se fundamentan en el Manifiesto Ágil y buscan un equilibrio entre "demasiado proceso" y "nada de proceso". Los cuatro pilares son:

1. **Individuos en interacciones por sobre procesos y herramientas**
 - Prioriza un equipo motivado con comunicación fluida pero con herramientas pobres
 - Es mejor que un equipo desorganizado con excelentes herramientas
 - El software es una actividad **humano-intensiva**
2. **Software funcionando por sobre documentación extensiva**
 - Se documenta solo lo que agrega valor al producto
 - La documentación se genera cuando sea necesaria
 - Prioriza versiones estables e incrementales del software
3. **Colaboración con el cliente por sobre negociación contractual**
 - El cliente debe estar comprometido con cada entrega
 - Reduce dependencia de documentación escrita
 - Minimiza fricción en relación equipo-cliente
 - Los contratos suelen imponer restricciones negativas
4. **Responder a cambios por sobre seguir un plan**
 - Los usuarios no pueden predecir todas las funcionalidades necesarias
 - Los entornos cambian constantemente (mercado, políticas)
 - Los equipos ágiles están abiertos a cambios rápidos

- Se valorizan la **inspección y adaptación** sobre planificación y control

Filosofía Ágil

Objetivo primordial: Entregar software de forma **frecuente en entornos cambiantes**

- Ciclo de vida **iterativo-incremental**: software se desarrolla como series de incrementos
- Basado en el concepto del **empirismo** (transparencia, inspección, adaptación)
- Los incrementos se crean cada **2-3 semanas** y se ponen a disposición del cliente
- Cliente inmerso en el proceso, ofreciendo **retroalimentación rápida**
- Se reduce documentación mediante **comunicaciones frecuentes informales**

Pilares del Empirismo:

- **TRANSPARENCIA:** Transforma conocimiento implícito en explícito. "No es mi código, es nuestro código"
- **INSPECCIÓN:** Detecta varianzas y problemas. Aprende cosas nuevas que surgen del trabajo
- **ADAPTACIÓN:** Mejora continua. Ajusta el proceso según resultados de inspección

Filosofía Lean

Definición: Maximiza valor generado al cliente con **mínimo consumo de recursos**. Todo esfuerzo que no cree valor = desperdicio.

Objetivo: Producir exactamente lo que el cliente quiere, cuando lo quiere, dónde lo quiere, a mínimo costo

7 Principios Lean (adaptados para software)

Nota Importante: Estos principios fueron adaptados del Sistema de Producción de Toyota por Mary y Tom Poppendieck en su libro "Lean Software Development". Los originales eran 7 desperdicios; aquí los transformamos en 7 principios constructivos.

PRINCIPIO 1: ELIMINAR DESPERDICIOS

Definición General:

Identificar y eliminar **cualquier actividad que consume recursos pero NO agrega valor al cliente final**. El 80% del valor está en el 20% de las funcionalidades, así que el 80% restante es potencial desperdicio.

Los 7 Desperdicios Específicos en Software:

1.1 - Características Extras (Overproduction)

- **Qué son:** Features desarrolladas que el cliente nunca solicitó o nunca usará
- **Cómo ocurren:** Especificación anticipada (Up Front Specification) donde se intenta predecir todo al inicio
- **Impacto:**
 - Tiempo desarrollo en funcionalidades no usadas
 - Tiempo testing en features innecesarias
 - Aumento de complejidad del código
 - Mayor mantenimiento futuro
 - Cliente paga por lo que no necesita
- **Ejemplo Real:** Cliente pide "carrito de compras" en mes 1, pero en mes 3 cuando se implementa, el método de pago cambió completamente. Feature obsoleta = retrabajo
- **Solución:** Diferir decisiones, entregas cortas (2-3 semanas), feedback temprano, principio YAGNI ("You Aren't Gonna Need It")

1.2 - Trabajo a Medias (Work In Progress - WIP)

- **Qué es:** Trabajo comenzado pero no terminado (User Stories, bugs, commits incompletos)
- **Cómo ocurre:** Prioridades cambian, urgencias aparecen, desarrollador abandona tarea a mitad de camino
- **Impacto:**
 - Context switching costoso (15-25 min por cambio de tarea)
 - Pérdida de conocimiento del problema
 - Riesgo de inconsistencias y errores

- Demora en entregas (nada se termina rápido)
- Métricas engañosas ("estamos 80% listos")
- **Por qué es malo:** Retomar trabajo requiere "ramp-up" (tiempo para entender qué se estaba haciendo). Ejemplo: Dev 1 hace 60% de feature, abandona. Dev 2 retoma 2 semanas después: debe entender, recordar contexto, testear trabajo previo
- **Gestión Binaria en Agile:** Una User Story está **LISTA o NO está LISTA** (no existe 50% listo)
- **Solución:** Limitar WIP en Kanban, completar antes de iniciar, sprints cortos, pull system

1.3 - Proceso Extra (Extra Processing)

- **Qué es:** Pasos en el proceso de desarrollo que NO agregan valor pero consumen tiempo/recursos
- **Ejemplos:**
 - Documentación exhaustiva que nadie lee
 - Reuniones innecesarias ("reunión porque es viernes")
 - Reportes automáticos que no se usan
 - Aprobaciones múltiples de la misma cosa
 - Herramientas innecesarias (5 tools cuando basta 1)
 - Ceremonias mal aplicadas (Daily Standup de 45 minutos)
- **Contraste:**
 - **Procesos Definidos (Cascada):** Documentación exhaustiva, aprobaciones múltiples, comités de cambio. Resultado: burocracia sin valor
 - **Procesos Empíricos (Agile):** Solo lo esencial, decisiones rápidas, mejora continua
- **Ejemplo Real:** Proceso actual = Dev escribe 2h → espera 1 día review → llena 1h formulario → presenta en comité (1 hora + espera) → QA testea 4h → llena Release Notes 1h → merge 10min = **1.5 días total** pero **2 horas de trabajo real en código**. Desperdicio: **~13 horas**

- **Solución:** Automatizar todo, documentación auto-generada, descartar procesos inútiles, preguntarse "¿Agrega valor al cliente?"

1.4 - **Búsqueda de Información (Waiting for Information)**

- **Qué es:** Tiempo gastado buscando información/documentación que DEBERÍA estar accesible
- **Impacto Real:**
 - Dev nuevo: "¿Dónde está documentación de la API?" - 30 min buscando
 - "¿Cuál es la versión en producción?" - 1 hora investigando
 - "¿Cómo configuro el ambiente?"
 - "¿Quién es owner de este módulo?"
 - Multiplicado por 20 devs = 10 horas perdidas
 - Multiplicado por 250 días/año = **1.2 años de un dev completo desperdiciado**
- **Relación con SCM (Software Configuration Management):**
 - **Sin SCM = CAOS:** Nadie sabe qué está donde
 - **Con SCM:** Trazabilidad (quién cambió qué y cuándo), versiones (qué código va con qué versión de BD), configuración clara, auditoría
- **Síntomas:** "Nadie sabe qué versión tenemos en producción", "El ambiente local de Dev A no funciona como el de Dev B", documentación en 5 lugares distintos
- **Solución:** Implementar SCM robusto (Git, Terraform, Helm), documentación única y actualizada, automatizar, trazabilidad completa

1.5 - **Defectos (Quality Issues)**

- **Qué es:** Errores introducidos en una etapa, descubiertos mucho después en otra etapa
- **Error vs Defecto:**
 - **Error:** Se introduce y se corrige en **misma etapa** (costo mínimo)
 - **Defecto:** Se introduce en una etapa, se descubre en otra (costo 10-100x mayor)

- **Costo Real:**

Defecto en **Desarrollo** (1 día después): 30 minutos
Defecto en **Testing** (2 semanas después): 11 horas
Defecto en **Producción** (cliente): 21 horas + reputación d
añada

- **Por qué ocurren:** **Testing tardío**, documentación pobre, comunicación deficiente, presión de tiempo, deuda técnica
- **Introduce TODOS los desperdicios anteriores** porque obliga retrabajo
- **Solución:** Calidad continua, TDD (tests desde inicio), automatización, code review, validar requerimientos temprano

1.6 - **Esperas** (Waiting / Delays)

- **Qué es:** Tiempo que un dev espera para poder continuar su trabajo
- **Tipos:**
 - Espera a aprobaciones (PR esperando 2 días en revisión)
 - Espera a otros devs (Dev A bloqueado esperando a Dev B)
 - Espera a infraestructura ("Espera a que ITOps prepare el servidor" - 1 semana)
 - Espera a feedback del cliente (2 semanas sin respuesta)
- **Impacto:** Dev a \$50/hora esperando 8 horas/día = \$400 perdidos/día × 5 días = \$2000/semana × 52 semanas = **\$104,000/año por dev**
- **En equipo de 10 devs con 50% esperas: \$520,000 perdidos al año** y software se entrega más lento
- **Relación con Equipos Multidisciplinarios:** Si todos pueden hacer todo, no hay esperas. Si especializados, uno espera al otro
- **Solución:** Equipos cross-functional, auto-servicio (dev provisiona su BD), async review, parallelizar, SLA rápidos (PR revisados max 4 horas)

1.7 - **Cambios de Tareas / Multitasking** (Task Switching)

- **Qué es:** Cambiar constantemente entre tareas diferentes (el MAYOR destructor de productividad)

- **Costo Científico:** Cerebro necesita **15-25 minutos** para recuperar "momentum" después de cambio
- **Ejemplo Real:**

1h de **trabajo** (EN REALIDAD):

- Task **A: 5 min** (luego cambio a **B**, pierdo **15 min** contexto)
- Task **B: 10 min** (luego cambio a **C**, pierdo **15 min** contexto)
- Task **C: 5 min** (luego vuelvo a **A**, pierdo **15 min** contexto)
- Task **A: 10 min**

RESULTADO: 30 min trabajo real + 30 min context switching = 50% productividad

- **En Software:** Desarrollo de software es trabajo intelectual complejo
 - Cambio frecuente = pérdida total de momentum
 - Si cambias cada 30 min, pierdes 50% del tiempo solo en cambios
- **Relación con Kanban:** WIP Limits **fuerzan concentración**
 - Máximo 2-3 cosas simult = no puedes multitaskear
- **Síntomas:** "Estoy en 3 projects simultáneamente", "Siempre ocupado pero no termino nada", interrupciones constantes
- **Solución:** Limit WIP, fokus time (2 horas sin interrupciones), termina una cosa antes de iniciar otra, Slack en silencio

PRINCIPIO 2: AMPLIFICAR EL APRENDIZAJE

Definición:

Transformar **conocimiento implícito en conocimiento explícito** para que todo el equipo lo comparta y aprenda continuamente.

Qué significa:

- Cada persona tiene **conocimiento en su cabeza** que nadie más sabe
- Ese conocimiento debe hacerse visible, documentarse, compartirse
- El equipo crece colectivamente, no solo individualmente

- Mejora continua basada en experiencias reales

Cómo Aplicarlo:

- **Documentación Viva:** Code comments, ADRs (Architecture Decision Records), wikis actualizados
- **Knowledge Sharing:** Pair programming, code reviews, presentaciones internas
- **Retrospectivas:** Lecciones aprendidas después cada sprint/release
- **Testing como Documentación:** Tests unitarios documentan cómo usar código
- **Cultura de Preguntas:** Junior puede preguntar al Senior sin miedo
- **Onboarding Estructurado:** Nuevos devs aprenden del existente

Impacto:

- Menos bus factor (si 1 dev se va, qué pasa?)
- Mejor toma de decisiones (más contexto compartido)
- Mejora continua (todos contribuyen con ideas)
- Retención de talento (devs quieren aprender)

Relación con Principios Ágiles:

- Principio 2 de Agile: "Recibir requerimientos aún en etapas finales"
- Principio 4: "Personas técnicas y no técnicas trabajando juntos todo el proyecto"

PRINCIPIO 3: EMBEBER LA INTEGRIDAD CONCEPTUAL

Definición:

La **calidad NO se negocia**. El **producto** debe ser **coherente, simple y satisfacer expectativas del cliente** como una **unidad integrada**.

Qué significa:

- No es sobre perfección, es sobre **consistencia interna**
- Todas las partes del producto trabajan juntas armónicamente
- Una decisión de arquitectura afecta el diseño completo

- El usuario experimenta el producto como un TODO, no partes separadas

Ejemplos de Integridad:

-  Interfaz consistente: todos los botones funcionan igual, naming coherente
-  Interfaz sin integridad: un button hace click, otro no; unos campos son required, otros no
-  Arquitectura íntegra: patrones de diseño consistentes, división clara de responsabilidades
-  Arquitectura sin integridad: código espagueti, cambios en un módulo rompen otro

Cómo Aplicarlo:

- **Code Standards:** Todos programan igual (revisar PRs estrictamente)
- **Architectural Reviews:** Decisiones grandes se toman considerando impacto global
- **Refactoring Continuo:** Mantener código limpio, no deuda técnica
- **User Experience Consistent:** Design system, no cada dev diseña diferente
- **Testing Exhaustivo:** Asegurar que todas las partes funcionan juntas

Por qué Importa:

- Cliente experimentará mejor producto, más fácil de usar
- Desarrolladores pueden mantener/extender sin sorpresas
- Menos defectos porque todo es predecible
- Más fácil onboarding de nuevos devs (patrón único)

PRINCIPIO 4: DIFERIR COMPROMISOS HASTA EL ÚLTIMO MOMENTO RESPONSABLE

Definición:

Postergar decisiones **lo máximo posible** para tener máxima información, pero **antes de que sea demasiado tarde** para cambiar.

Por Qué:

- Incertidumbre en el inicio = información incompleta

- Esperar permite tener más datos antes de decidir
- Decisiones tardías = mejor informadas = mejor resultado
- Pero esperar demasiado = imposible implementar

Ejemplo Real:

MES 1: Cliente pide "arquitectura con BD relacional"

- Si decidimos **YA:** Implementamos todo con PostgreSQL
- **MES 3:** Cliente revela que datos son no-estructurados
- **PROBLEMA:** Refactorizar todo es costoso

Alternativa **LEAN:**

MES 1: Cliente pide "arquitectura flexible"

- No especificamos **BD** aún, solo interfaz
- **MES 2:** Exploramos datos, vemos que son no-estructurados
- **MES 3:** Decidimos MongoDB, es fácil cambiar porque fue planeado
- **RESULTADO:** Mejor decisión, menos retrabajo

Relación con Product Backlog:

- Nunca tienes 100% de requerimientos especificados
- Se completan a medida que aprendes
- Esto es **intencional y bueno**

Cómo Aplicarlo:

- **Product Backlog Vivo:** Solo los siguientes sprints bien definidos, resto flexible
- **Spike Stories:** Investigar opciones antes de decidir
- **Feature Flags:** Implementar sin decidir si desplegar
- **Reversible vs Irreversible:** Tomar irreversibles tarde, reversibles temprano
- **MVPs:** Validar con cliente rápido antes de full-scale

Equilibrio Delicado:

- Demasiado pronto: Información incompleta, mala decisión
- Demasiado tarde: No hay tiempo para implementar

- "Último momento responsable" = justo antes de ser demasiado tarde
-

PRINCIPIO 5: DAR PODER AL EQUIPO

Definición:

Otorgar **libertad de acción y poder de decisión** al equipo. Equipos motivados, capacitados y autoorganizados entregan mejor software.

Qué significa:

- No micromanagement
- El equipo decide cómo resolver problemas
- Confianza en que harán lo correcto
- **Responsabilidad compartida**

Requisitos:

- **Multifuncionalidad:** **Todos pueden hacer todo** (no especialización extrema)
- **Autogestión:** **El equipo se organiza solo**, sin jefe dictando tareas
- **Capacitación:** **Todos saben qué hacen y por qué**
- **Motivación:** **Ambiente de trabajo positivo**, no castigo

Cómo Aplicarlo:

- **Decisiones Locales:** Dev decide técnica de implementación, no manager
- **Delegación:** Líder delega decisiones, no solo tareas
- **Mentoring:** Entrenar líderes dentro del equipo
- **Responsabilidad:** El equipo es dueño del código, no "culpables"
- **Confianza:** No revisiones exhaustivas, confianza en competencia

Reto Importante:

- **"Talón de Aquiles" de Agile y Lean:** Pocas veces se logra en práctica
- Managers acostumbrados a control tradicional luchan soltarlo
- Algunos equipos no están preparados para tanta libertad
- Requiere cambio cultural profundo

Impacto:

- Mejor moral del equipo (no se sienten controlados)
- Decisiones más rápidas (no esperan aprobaciones)
- Mejor software (el que lo hace lo sabe mejor)
- Retención (devs quieren autonomía)

PRINCIPIO 6: VER EL TODO (Visión Holística)

Definición:

Tener **visión integral** del sistema, no optimizar partes a costa del todo. Los objetivos locales deben alinearse con objetivos globales.

Por Qué:

- Fácil optimizar un subsistema (ej: hacer testing muy lento pero exhaustivo)
- Pero eso perjudica el sistema completo (proyecto se entrega tarde)
- Necesario ver CÓMO se conectan todas las partes

Ejemplos:

✗ OPTIMIZACIÓN LOCAL (Sin Ver el Todo):

- Team A optimiza: "Aceleramos dev con menos testing"
- Resultado: Más bugs llegan a producción
- Team B: "Necesitamos más QA para encontrar bugs"
- Resultado: Delivery se ralentiza mucho

✓ OPTIMIZACIÓN GLOBAL (Viendo el Todo):

- Todos los teams: "Necesitamos optimizar TIME-TO-MARKET"
- Decisión: TDD desde inicio (mejor balance entre velocidad y calidad)
- Resultado: Menos bugs, entregas más rápidas

Cómo Aplicarlo:

- **Métricas Globales:** Medir value delivered, no lines of code por dev
- **Alineación de Objetivos:** Team goals = Company goals
- **Cross-team Communication:** No silos entre diferentes áreas
- **Trade-offs Conscientes:** Si optimizo A, ¿qué pasa con B?

- **Retrospectivas Juntas:** Teams aprenden juntos de errores

Relación con Organizaciones:

- Problemas surgen cuando cada departamento optimiza su métrica
- Dev quiere código rápido, QA quiere máxima cobertura
- Sales promete features imposibles, Dev sufre
- Solución: **Objetivos compartidos**

PRINCIPIO 7: ENTREGAR LO ANTES POSIBLE 🚀

Definición:

Entregar **software funcionando lo más rápido posible** para que cliente obtenga valor pronto y pueda dar feedback temprano.

Por Qué Importa:

- **Mercado cambia:** Esperar 6 meses = requisitos obsoletos
- **Feedback temprano:** Cliente usa software real, no especificación
- **Reducir riesgo:** Lanzar MVP rápido prueba concepto
- **Competencia:** Primero al mercado = ventaja

Cómo:

- **Acotar ciclos de desarrollo:** De 6 meses a 2 semanas
- **Entregas incrementales:** Pequeño incremento cada 2 semanas, NO feature gigante cada 6 meses
- **MVP (Minimum Viable Product):** Versión más pequeña que agrega valor
 - Ej: Uber en 1 ciudad, no 50 ciudades primero
 - Ej: Twitter sin retweets/favorites, solo tweets
- **Continuous Deployment:** Nuevo código a producción cada día (no cada 6 meses)

Ejemplo Real:

APROACH TRADICIONAL:

- Mes 0-2: Especificación completa
- Mes 2-4: Desarrollo completo

- Mes 4-5: Testing exhaustivo
- Mes 5-6: Despliegue
- Mes 6: Cliente ve versión **INCOMPLETA** de requisito
- **RESULTADO**: Requisitos cambiaron en mes 6, retrabajo

APROACH LEAN:

- Semana 1-2: **MVP** con 20% features core
- Semana 2: Despliegue a cliente
- Semana 3-4: Cliente usa, da feedback
- Semana 5-6: Agregar 30% más features basado en feedback real
- Semana 6: Nuevo despliegue
- **RESULTADO**: Cliente happy, menos retrabajo, más incremental

Relación con Principio 1 (Eliminar Desperdicios):

- Entregas rápidas = cliente ve cosas no usadas temprano
- Puede decir "no necesito esto" = menos desperdicio
- Vs esperar 6 meses, desarrollas todo, descubres que 80% no sirve

RESUMEN VISUAL: Los 7 Principios

#	Principio	Foco	Beneficio
1	Eliminar Desperdicios	Eficiencia	Menos costo, más valor
2	Amplificar Aprendizaje	Conocimiento	Equipo crece, menos errores
3	Integridad Conceptual	Calidad	Producto coherente, user happy
4	Diferir Decisiones	Información	Mejores decisiones, menos riesgo
5	Dar Poder al Equipo	Autonomía	Decisiones rápidas, buen moral
6	Ver el Todo	Alineación	Objetivos comunes, menos conflictos
7	Entregar Rápido	Velocidad	Feedback temprano, market

#	Principio	Foco	Beneficio
			advantage

Relación Lean-Ágil

- Lean es **anterior al Manifiesto Ágil**
- Algunos principios ágiles heredan fundamentos de Lean
- Ambos: orientados al cliente, máximo valor, equipos motivados, flexibilidad, mejora continua
- **Síntesis:** Lean proporciona la filosofía, Ágil la metodología de implementación

Kanban

Definición: Enfoque para la **gestión de formas de trabajo** para obtener **mejora continua**

Filosofía: "Empieza por donde estés" - **introduce mejoras graduales a proceso existente** (reduce resistencia al cambio)

Origen: Estudios de Toyota para optimizar almacenamiento en supermercados

Aprovecha los Principios Lean:

- Definir valor desde perspectiva del cliente
- Limitar WIP (Work In Progress)
- Identificar y eliminar desperdicios y barreras en flujo
- Cultura de mejora continua

Prácticas de Kanban

1. Visualización del Trabajo

- Tablero Kanban = sistema de flujo
- Divide trabajo en piezas (US, Features, bugs, etc.)
- Tarjetas señalizadoras (kan-bans) en tablero
- **Columnas = etapas del proceso**
- Flujo de **izquierda a derecha**

- Dos tipos de columnas:
 - **Producción:** piezas en trabajo
 - **Acumulación:** piezas listas para siguiente etapa (sistema pull)

2. Limitar WIP (Work In Progress)

- Asignar límites concretos por etapa
- Sistema pull (arrastre): trabajo se "tira" cuando hay capacidad
- Evita atascamientos y cuellos de botella
- Tener demasiado WIP = desperdicio de tiempo y dinero

3. Gestionar el Flujo de Trabajo

- Trabajo fluye continuamente (sin concepto de iteración)
- Analizar flujo: cuellos botella, recursos ociosos, tiempos entrega, tiempos espera
- Flujo **continuo e ininterrumpido**

4. Hacer Explícitas las Políticas

- Políticas **escasas, simples, bien definidas y visibles**
- Fácilmente modificables si producen efectos contraproducentes
- DoD (Definition of Done) definidas y publicadas
- Mejorar continuamente

5. Mejorar y Evolucionar

- Identifica proceso → visualiza → propone mejoras
- Cambios graduales a lo largo del tiempo
- No cambios significativos de golpe

6. Circuito de Retroalimentación

- Parte esencial para proceso controlado
- Frecuencia depende del contexto
- Demasiado frecuentes = cambios innecesarios
- Muy infrecuentes = bajo rendimiento prolongado

¿Cómo Aplicar Kanban?

1. Entender proceso de desarrollo actual
2. Identificar unidades de trabajo (US, CU, defectos)
3. Identificar clases de servicio (diferentes políticas/DoD)
4. Visualizar flujo diseñando tablero Kanban
5. Definir políticas para cada clase de servicio (WIP, colores, capacidad, fechas entrega)
6. Identificar y resolver cuellos de botella

Métricas de Kanban

- **Cycle Time:** tiempo desde que equipo comienza hasta entrega
- **Lead Time:** tiempo desde solicitud cliente hasta entrega
- **Touch Time:** tiempo efectivo trabajando (sin esperas)
- **Eficiencia de Ciclo:** $\text{Touch Time} / \text{Lead Time}$ (cercano a 1 = eficiente)

Valores de Kanban (Base: RESPETO)

1. **Transparencia:** Compartir información abiertamente mejora flujo
2. **Equilibrio:** Balance entre diferentes aspectos para efectividad
3. **Colaboración:** Trabajar juntos es el corazón del método
4. **Foco en el cliente:** Conocer objetivo del sistema
5. **Flujo:** Realización del trabajo es el flujo de valor
6. **Liderazgo:** Inspirar a otros a la acción en todos los niveles
7. **Entendimiento:** Conocimiento de sí mismo (individual y organización)
8. **Acuerdo:** Compromiso dinámico para mejorar juntos
9. **Respeto:** Base sobre la cual reposan los demás valores

Principios Directores:

- **Sostenibilidad:** Encontrar foco sostenible y mejora

- **Orientación al servicio:** Rendimiento y satisfacción del cliente
- **Supervivencia:** Mantener competitividad y adaptabilidad

Principios Fundacionales:

Principios de Gestión de Cambio:

1. **Empezar con lo que estés haciendo ahora**
 - Entender procesos actuales
 - Respetar roles y responsabilidades actuales
2. **Acordar en buscar la mejora** a través del cambio evolutivo
3. **Fomentar el liderazgo en cada nivel** de la organización

Principios de Despliegue de Servicios:

1. **Entender necesidades y expectativas de clientes** y focalizarse en ellas
2. **Gestionar el trabajo:** permitir que gente se auto-organice alrededor de tareas
3. **Evolucionar políticas para mejorar resultados** hacia cliente y negocio

Diferencia SCRUM vs KANBAN

Aspecto	SCRUM	KANBAN
Tiempo	Iteraciones fijas	Tiempo variable
Compromiso	Equipo asume compromiso	Compromiso opcional
Métrica default	Velocidad	Lead Time
Equipos	Multifuncionales	Multifuncionales o especializados
Diagramas	Burndown obligatorio	No prescrito
Limitación WIP	Indirecta (por sprint)	Directa por etapa
Agregar trabajo	No durante sprint	Siempre que hay capacidad
Estimaciones	Obligatorias	Opcional
Pizarra	Se limpia cada sprint	Persistente
Roles	3 prescritos	No prescritos
Priorización	Product Backlog priorizado	Opcional
Tamaño features	Debe completarse en sprint	No prescrito

Nota: Kanban **no tiene concepto de iteración ni proyecto** - trabajo fluye continuamente

Frameworks para Escalar SCRUM

Nexus: Framework para 3-9 equipos Scrum sobre un único Product Backlog común

Responsabilidades en Nexus:

- **Nexus Integration Team:** Asegura incremento integrado al menos 1 vez por sprint
- **Product Owner:** Maximiza valor, gestiona Product Backlog
- **Scrum Master:** Asegura comprensión del marco Nexus
- **Integrantes NIT:** Miembros Scrum Teams que mejoran capacidad de entregar

Eventos Nexus:

- **Nexus Sprint:** Produce Integrated Increment
- **Refinamiento entre equipos:** Reduce/elimina dependencias entre equipos
- **Nexus Sprint Planning:** Coordina actividades de todos los Scrum Teams
- **Nexus Daily Scrum:** Discute problemas integración
- **Nexus Sprint Review:** Retroalimentación sobre Integrated Increment
- **Nexus Sprint Retrospective:** Mejora calidad y eficacia del Nexus

Artefactos Nexus:

- **Product Backlog:** Único, contiene todo lo necesario. Compromiso: **Objetivo del Producto**
 - **Nexus Sprint Backlog:** Compuesto de Objetivo Sprint Nexus + elementos PB de cada Scrum Team
 - **Integrated Increment:** Suma actual de trabajo integrado completado. Compromiso: **Definición de Terminado**
-

Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

Testing en Ambientes Ágiles

Definición de Testing:

Proceso mediante el cual se somete a software o componente a **condiciones específicas** para determinar si es válido según requerimientos

- Actividad **destructiva** que busca encontrar defectos
- Proceso para ejecutar programa con **intención de encontrar fallas**
- Software debe ser **predecible y consistente**
- Testing es **parte del QA**, forma parte del control de calidad

Diferencia Testing vs QA:

- **Testing:** Se hace cuando producto ya está construido
- **QA:** Se aplica en todo ciclo de vida (buenas prácticas implementación)

Actividades exitosas de Testing: Encuentran defectos

Nota: Software con **alta calidad** (por QA) = **difícil encontrar defectos** = testing "no exitoso" (paradoja)

- **Bajo nivel de calidad** = alto costo de retrabajo, muchas idas y vueltas

Testing es COSTOSO: 30-50% del valor del producto

- Necesario lograr **cobertura óptima** (costo-beneficio)
- Nunca 100% de pruebas por tiempo y costos

Principios del Testing

1. **Actividad destructiva** que encuentra defectos asumidos
 - Actitud negativa tratando de demostrar que algo es incorrecto
2. **Testing exitoso = encuentra defectos**
3. **Costo está entre 30-50% del valor del producto**
4. **Pone en evidencia defectos pero NO agrega calidad ni garantiza ausencia de errores**
5. **Un desarrollo exitoso puede llevar a Testing no exitoso**

- Menos defectos para encontrar
6. **Testing es necesario siempre**
 - Suposición: siempre habrá defectos por encontrar
 - Característica inherente de productos desarrollados por equipos
 7. **Puede empezar antes de codificación**
 - Necesita requerimientos para armar casos de prueba
 8. **NO asegura producto de calidad ni que proceso fue de calidad**
 9. **Agrupamiento de defectos (Regla de Pareto)**
 - 80% defectos en 20% funcionalidad
 - Priorizar pruebas en conjunto acotado de funcionalidades

Mitos del Testing

✗ "Testing demuestra que NO hay errores"

✗ "Propósito es demostrar que programa funciona correctamente"

✓ **Verdad:** Objetivo es encontrar y remover defectos, revelar calidad, brindar confianza

- Partimos de suposición que software **contiene defectos**
- Realizamos testing para **encontrar mayor cantidad posible**

Dato importante: Un programa puede hacer lo que se supone debe hacer y aún contener defectos

- Error está presente si NO hace lo que debe
- **TAMBIÉN** si hace lo que **NO debe**

¿Cuánto Testing es Suficiente?

Testing exhaustivo = **imposible** por tiempo requerido

Momento de dejar Testing: Depende del **nivel de riesgo/costo** asociado al proyecto

Criterios de Aceptación (decidir cuándo terminar):

- Costos
- % de tests corridos sin fallas

- Inexistencia de defectos de determinada severidad
- Pasa exitosamente conjunto de pruebas diseñado + cobertura estructural
- "Good Enough:" Cierta cantidad de fallas no críticas es aceptable
- Defectos detectados similar a cantidad estimada

Criterio de aceptación: Define y negocia con PO (ágil) o Líder Proyecto (tradicional) cuántos defectos son aceptables para terminar

Conceptos Importantes

Defecto vs Error

Error: Detectado y corregido en **misma etapa**

Defecto: Error trasladado a **etapa posterior** donde se introdujo

Diferencia: Momento en que se detectan y solucionan

Relación con Testing: Testing encuentra **defectos** (errores de etapas anteriores detectados en prueba)

Nota: Ambos pueden generar o no fallas (mal funcionamiento)

Defecto - Características:

Severidad (carácter técnico, asignado por tester):

- **Bloqueante:** NO permite continuar ejecución
- **Crítico:** Sistema funciona pero funcionalidad testeada tiene defecto crítico
- **Mayor:** Funcionalidad funciona pero no correctamente
- **Menor:** Funciona correctamente pero con advertencias/errores bajos
- **Cosmético:** Formato fechas, números, distribución GUI

Prioridad (impacto funcional para negocio, ordena atención):

- Urgencia
- Alta
- Media
- Baja

Nota: Independientes entre sí. Ej: defecto cosmético = baja prioridad en inventarios pero alta en empresa marketing

Casos de Prueba

Definición: Secuencia de pasos a seguir para obtener resultado esperado

Importancia: Artefacto **MÁS IMPORTANTE del Testing**

- Se hace UNA VEZ pero sirve para infinitas pruebas
- Deben obtener MISMO resultado esperado

Objetivo: Descubrir defectos

- Minimizar cantidad de casos de prueba
- Maximizar cantidad de defectos encontrados

Origen: Salen de requerimientos

- Permiten verificación Y validación

Composición:

- **Objetivo:** Lo que se desea controlar
- **Condiciones de prueba:** Datos entrada + entorno necesarios
- **Resultado esperado:** Qué debe ocurrir

Ciclos de Prueba

Definición: Ejecución de conjunto de casos de prueba en versión determinada del producto

- Generalmente **2 ciclos**
- **Ciclo 0:** Siempre manual, configuración inicial
- **Ciclo 1+:** Pueden automatizarse

Si detectan defectos: Producto vuelve a desarrollo para corrección

Indicador: Alta cantidad ciclos = QA no bueno

- Indicador de retrabajo excesivo
- Incrementa costos de desarrollo
- Necesario revisar calidad de proceso y producto

Ciclo de Pruebas con Regresión:

- Ideal: ejecutar exhaustivamente TODOS casos en CADA ciclo
- Menos práctico pero estadísticamente mejor
- Alternativa: ejecutar todos en ciclo 0, luego solo los con defectos hallados
 - Ágil pero riesgo: arreglar defecto introduce nuevos errores en otras partes
 - Solución: cuando todos "corregidos", ejecutar todos casos (como regresión) para confianza

Automatización: Permite tantos ciclos con regresión como se deseen

- Costo inicial de automatización pero se reutiliza N veces
- Recomendado si core de negocio no es software

Niveles de Prueba

Determinan **foco/granularidad** de la prueba a ejecutar

Orden: pequeños componentes → pruebas mayor granularidad (opuesto al desarrollo)

1. Pruebas Unitarias

- Realizada por **desarrollador**
- Incluida en **DoD**
- Pruebas sobre **componente independiente**
- Acceso a **código fuente**
- Herramientas: automatización, depuración
- Errores se reparan apenas se encuentran, sin registro formal

2. Pruebas de Integración

- Verificar funcionamiento de **2+ componentes** que se relacionan
- Interacción a través de **interfaces (boundaries)**
- Resultado: **build** (CI) → enviado a equipo testing

- Prueba **incremental**: top-down (general a específico) o bottom-up (específico a general)

3. Pruebas de Sistema o Versión

- Testeos sobre **versión de incremento/producto**
- Realizadas por **personas ajenas al desarrollo** (psicológicamente necesario)
- **Proceso sistemático y metodológico**
- Basadas en **casos de prueba**
- **Emular entorno idéntico a real**
- Pruebas del funcionamiento **real y cotidiano**
- Abarca requerimientos **funcionales y no funcionales**

4. Pruebas de Aceptación de Usuario

- Se realizan en **despliegue**
- Debería realizarlas el **usuario**
- Verificar que se cumple con lo requerido
- **Objetivo NO es encontrar fallas sino generar comodidad y confianza**
- Comprende: pruebas en laboratorio (alfa) + ambiente real (beta)
- En teoría: usuarios arman pruebas. En práctica: Testing las arma
- En teoría: gerente testing, líder proyecto, empleados funcionales. En Agile: todos miembros equipo (Sprint Review)

Modelo en V

Determina **cuándo se pueden realizar actividades** según etapa de desarrollo

Muestra que pruebas en **orden granularidad inverso** al proceso de desarrollo

Relación:

- **Desarrollo:** Granularidad gruesa (requerimientos) → fina (clases/módulos)
- **Pruebas:** Granularidad fina (componentes/unitarias) → gruesa (sistema/aceptación)

Mapeo:

- Test Aceptación Usuario ↔ Requerimientos usuario y negocio
- Test Sistema ↔ Requerimientos software
- Test Integración ↔ Especificación del programa
- Test Componentes/Unitario ↔ Especificación de componentes

Ambientes de Testing

Recursos hardware y software necesarios para trabajar con producto

1. Desarrollo

- Hardware y software necesarios (librerías, extensiones, IDEs, compiladores)
- Pruebas unitarias (y a veces integración)

2. Pruebas

- Ambiente para testers (sin acceso desarrolladores)
- Normalmente: pruebas de sistema

3. Preproducción

- Mismas características que producción
- Comprobar que producto funcionará correctamente al desplegarse
- Pruebas de aceptación
- **Problema:** Difícil/imposible replicar configuraciones hardware (costoso)

4. Producción

- Configuración hardware/software de usuarios finales
- **NO se realizan pruebas aquí** (graves consecuencias)
- En teoría, versión cumple Definition of Done

Proceso de Pruebas

Testing es **proceso definido** (durante todo ciclo vida)

Etaa Ad Hoc: Testing sin proceso definido, sin trazabilidad

Proceso Standard:

1. Planificación

- Determina cómo se incluirá Testing en plan proyecto
- Qué recursos, riesgos, criterio aceptación, entornos, quién realiza, cuándo
- **Output:** Plan de Pruebas (que contiene riesgos, objetivos, estrategia, recursos, criterio aceptación)

2. Diseño (Identificación y Especificación de Casos de Prueba)

- Revisando bases de planificación Testing
- Identifican datos necesarios, diseñan y priorizan CP
- Definen entorno, cómo llevarse a cabo, por quién, cuándo
- Analiza si requerimientos son testeables
- Define si usa regresión

3. Ejecución

- Ejecutan CP, registran y comparan resultados
- Generan reporte de defectos (defectos encontrados, condiciones, entornos)
- Automatizar máximo posible (ahorran tiempo/costo)
- Incluye: creación datos, automatización, implementar/verificar ambiente, ejecutar CP, registrar resultados

4. Análisis de Fallas (Evaluación y Reporte)

- Seguimiento corrección defectos hasta cerrar todos CP
- Evalúa criterio aceptación
- Reporta resultados a interesados
- Verifica entregables y defectos corregidos
- Evalúa cómo resultaron actividades testing y lecciones aprendidas
- Confecciona informe de reportes

5. Fin de Pruebas

- Empresas maduras: dejan probar cuando NO hay bloqueantes, críticos, mayores, y menores/cosméticos son muy pocos (en Release Notes)
- Confecciona informe final (opcional)

Artefactos de Testing

1. Plan de Pruebas

- Análogo al plan de proyecto (pero separado)
- Define: datos necesarios, recursos, herramientas, si usa regresión, etc.
- Confecciona SIN código, solo con requerimientos

2. Casos de Prueba (MÁS IMPORTANTE)

- Secuencia de pasos a seguir
- Conjunto de acciones para lograr resultado esperado
- Condiciones de inicio explícitas
- Datos utilizados en escenario
- **Característica VITAL: REPRODUCIBLES**
- Si defecto no reproducible a través CP = NO es defecto
- Se derivan de: documentos cliente, información relevada, requerimientos, especificaciones programación, código

3. Reporte de Incidentes o Defectos

- Después ejecución CP
- Indicar cuáles CP pasaron y cuáles encontraron defectos
- Cuáles fueron esos defectos y cómo reproducirlos
- Permite desarrolladores corregir

4. Informe Final

- Contiene métricas e información final del incremento
- Reporta: ciclos, cantidad errores por ciclo, métodos, tipos prueba utilizados

- Único artefacto negociable en organizaciones informales
- Utilidades estadísticas

Estrategias de Prueba: Caja Negra

Definición: No se dispone estructura interna, se analizan funcionalidades como "caja negra" (entradas y salidas)

Proceso:

- Ingresar datos determinados
- Comparar resultados obtenidos con resultados esperados
- Testing exhaustivo de entrada (toda entrada posible, incluso inválidas)
- En transacciones: considerar TODAS datos posibles Y secuencias previas

Métodos Basados en Especificaciones:

1. Partición de Equivalencias

- **Proceso sistemático:** Identificar clases equivalencia
- Clases = subconjuntos datos que producen resultado equivalente

Procedimiento:

1. Identificar condiciones externas entrada/salida
2. Identificar subconjuntos valores posibles (válidos e inválidos) que produzcan resultados equivalentes
3. Subconjuntos inválidos = asociados a mensajes error
4. Validar: unión todos subconjuntos = universo condición externa (teoría conjuntos)
5. Armar CP tomando de cada condición externa un solo valor particular de cada subconjunto

Consideraciones al diseñar CP:

- **Prioridad:** Alta = caminos felices o errores críticos | Baja = validaciones
- **Nombre:** Describir escenario de forma clara
- **Precondiciones:** Características que deben cumplirse (valores concretos)

- **Pasos:** Siempre arrancan seleccionando opción que desencadena funcionalidad ("El usuario ingresa...")
- **Resultado esperado:** Concreto, específico, relacionado con paso que produce

2. Análisis de Valores Límites

- Variante de Partición Equivalencias
- Utilizar límites o valores de borde de clases equivalencia
- Para definición de CP

Métodos Basados en Experiencia:

1. Adivinanza de Defectos

- Basado en intuición y experiencia
- Identificar pruebas que probablemente expongan defectos
- Elaborar lista defectos posibles y probar a partir de esa lista

2. Testing Exploratorio

- Mientras prueba software, aprende a manejarlo
- Junto con experiencia y creatividad, genera nuevas pruebas
- Ejecuta mientras explora

Estrategias de Prueba: Caja Blanca

Definición: Se dispone del código (puede ser pseudocódigo o diagrama flujo)

Permite: Diseñar CP maximizando cantidad defectos con menor cantidad CP posibles

Limitaciones:

1. Número de caminos lógicos únicos puede ser sumamente grande (mucho tiempo probar todo)
2. Pruebas camino exhaustivas NO aseguran que programa sea correcto para especificaciones
 - NO detecta caminos faltantes ni errores datos sensibles

Coberturas (forma de recorrer caminos del código):

1. Cobertura de Enunciados (Caminos Básicos)

- Objetivo: encontrar TODOS caminos independientes que deben recorrerse al menos UNA VEZ

Métrica: Complejidad Ciclomática (M)

- Representa cantidad caminos independientes de funcionalidad
- Límite inferior número CP que construir para ejecutar TODAS instrucciones al menos 1 vez

Cálculo:

- $M = E - N + 2 * P$ (E = aristas, N = nodos, P = componentes conexos)
- O: $M = \text{regiones cerradas} + 1$

Procedimiento:

1. Representar funcionalidad a través grafo flujo
2. Calcular complejidad ciclomática
3. Definir conjunto mínimo caminos independientes (NO son CP)
4. Diseñar y ejecutar CP que permitan ejecución de cada camino
5. Ejecutar CP y comprobar resultados sean esperados

Clasificación por Complejidad (Heurística):

- **1-10:** Programa simple, sin mucho riesgo
- **11-20:** Más complejo, riesgo moderado
- **21-50:** Complejo, programa de alto riesgo
- **50+:** Programa no testeable, muy alto riesgo

2. Cobertura de Sentencias

- **Sentencia:** Cualquier instrucción (asignación variable, invocación métodos)
- Estructuras de control $\neq$ sentencias
- Objetivo: buscar cantidad MÍNIMA CP para ejecutar TODAS sentencias

3. Cobertura de Decisión

- **Decisión:** Estructura de control (IF, WHILE, etc.)
- Objetivo: cantidad MÍNIMA CP para evaluar TODAS ramas de decisiones

- Asegura que código tome cada rama correctamente
- En decisión IF: siempre 2 CP (rama true y false)

4. Cobertura de Condición

- **Condición:** Evaluaciones lógicas dentro decisión unidas por operadores lógicos
- Objetivo: cantidad MÍNIMA CP para evaluar CADA condición en valor verdadero Y falso
- Independientemente de rama que tome decisión

5. Cobertura de Decisión-Condición

- Objetivo: evaluar ramas verdaderas/falsas de decisiones Y valores verdaderos/falsos de condiciones

6. Cobertura Múltiple

- Objetivo: evaluar **combinatorio** de TODAS condiciones en todos valores verdad posibles
- Escenarios donde decisiones son dependientes entre sí

Tipos de Prueba

1. Smoke Test

- Valida que NO haya fallas groseras de gran magnitud
- Se realiza **antes de comenzar ciclo 0**
- Ahorra tiempo: si encuentra error, algo aún por corregir
- Consiste en **corrida rápida** para ver si producto está en condiciones de pasar al ciclo de prueba

2. Testing Funcional

- Controla que software se comporte como especificado en documentación
- Cumple con funcionalidades y características definidas
- Basado en **requerimientos funcionales y proceso de negocio**

Tipos:

- **Basado en requerimientos:** Prueba requerimientos específicos (una funcionalidad sola)
 - Utiliza requisitos definidos en ERS o acuerdos pruebas usuario
 - Criterios aceptación de US
- **Basado en proceso de negocio:** Prueba **proceso completo**
 - Ej: en venta se prueba búsqueda artículo, selección, facturación

3. Testing No Funcional

- Se basa en **cómo trabaja el sistema** (enfoque en el "como", no el "qué")
- Pruebas más complejas por dependencia al entorno del sistema
- Debe ser lo más parecido posible al del cliente
- Algunas características NO pueden probarse (ancho banda internet, seguridad en cierta situación)
- Pueden solucionarse con pruebas aceptación

Incluye varios tipos:

- **Performance:** Ver tiempo respuesta (escenario esperado vs tiempos reales) y concurrencia. DEBEN pasar
- **Carga:** No solo mira performance, mira comportamiento dispositivos hardware (procesadores, discos) y comunicaciones
- **Stress:** Forzar sistema para que falle. Ver tiempo espera para probar nuevamente y robustez (tiempo recuperación)
- **Mantenimiento:** Ver si producto está en condiciones de evolucionar. Controla documentación, manual configuración, facilidad corregir defectos
- **Usabilidad:** Que sea cómodo para usuario
- **Portabilidad:** Prueba en distintos entornos acordados con cliente
- **Fiabilidad:** Probar que podemos depender del sistema. Resultados obtenidos, seguridad física software
- **De interfaz de usuario:** GUI más complejas que interfaces comandos
- **De configuración**

4. Test Driven Development (TDD)

- **Tipo de proceso** en que nos enfocamos en construir **primero la prueba unitaria** cuando tenemos requerimientos
- Luego se codea el componente
- Si pienso en pruebas después = **menos errores**
- **Fundamento:** Si no tengo claro qué debo hacer, no puedo crear pruebas para eso

Técnicas:

- **Test First Development:** Primero se escriben pruebas unitarias de característica a implementar
 - Definidas las pruebas, se piensa en codificación para que pruebas ejecuten con éxito
 - Obliga desarrollador a **modularizar** código (métodos/clases con única responsabilidad)
 - Pueden incluirse pruebas de integración también
- **Refactoring:** Reestructurar código existente SIN modificar comportamiento
 - Mejora aspectos no funcionales
 - Objetivo: mejorar claridad código y reducir complejidad
 - Hacer código más **mantenible**

Revisiones Técnicas

Verificación y Validación

Concepto: Proceso de **ciclo de vida completo**

- Inicia con revisiones de requerimientos y continúa con revisiones diseño, inspecciones código hasta prueba

Definiciones:

- **Validación:** ¿Estamos construyendo el **producto correcto**?
- **Verificación:** ¿Estamos construyendo el **producto correctamente**?

Conceptos relacionados:

- **Falla:** Error en un producto de trabajo
- **Producto de trabajo:** Salida de cualquier actividad correspondiente al ciclo vida de desarrollo

¿Por qué existen fallas?

- Problemas de comunicación
- Limitaciones de memoria

Principios:

- **Prevenir es mejor que curar**
- **Evitar es más efectivo que eliminar**
- **La retroalimentación enseña efectivamente**
- **Priorizar lo rentable**
- **Olvidarse de la perfección, no se puede conseguir**
- **Enseñar a pescar, en lugar de dar el pescado**

Dos aproximaciones complementarias:

- Revisiones técnicas
- Pruebas de Software

Peer Review

Definición: Actividad realizada por un **colega** cuyo propósito es **mejorar calidad software** mediante detección temprana de errores en **cualquier artefacto**: código, requerimientos, diseño, arquitectura, riesgos, estimaciones, planes, etc.

Características:

- Proceso **estático** de validación y verificación
- **NO corrige errores** (solo identifica)

Objetivo:

- Introducir concepto de verificación y validación
- Busca **evitar retrabajo**
- **Motiva a realizar mejor trabajo**

Principio clave:

- **NUNCA** se pone en juicio el **autor** del artefacto, solo el **artefacto en sí**
- Necesario que se revelen TODOS errores entre miembros equipo
- Desarrollar cultura de trabajo que brinde **apoyo**, no buscar culpables

Contexto:

- Actividad que **transcendió cualquier metodología y filosofía**
- Agile la utiliza para transformar auditorías en peer reviews (sin alguien externo al equipo)
- Ej: programador Jr solicita revisión Sr sobre patrón diseño arquitectónico

Ventajas:

- Pueden descubrirse **muchos errores**
- Pueden inspeccionarse **versiones incompletas**
- Pueden considerarse **otros atributos de calidad**

Desventajas:

- Difícil introducir inspecciones formales
- Sobrecargan costos al inicio, ahorros después de experiencia
- Requieren tiempo para organizarse ("parece" ralentizar desarrollo)

Tipos de Revisiones Técnicas

1. Walkthrough (Recorrido) - INFORMAL

Definición: Técnica de análisis **estático** donde diseñador o programador dirige miembros equipo de desarrollo y otras partes interesadas a través de producto software

- Participantes formulan preguntas y realizan comentarios sobre posibles errores, violación estándares, otros problemas

Características:

- **NO existe proceso formal**
- Consiste en **reuniones informales** de colegas
- Se debaten correcciones a aplicar al producto trabajo

- **NO hay control del proceso**

Objetivos:

1. **Mínima Sobrecarga**
2. **Capacitación de Desarrolladores**
3. **Rápido retorno**

Métricas: NO obtiene (para aprender y dejar registros)

En Agile: Una de técnicas más elegidas

- Junta de revisión entre colegas después de completar cada iteración
- Revisión rápida donde pueden exponerse conflictos y problemas calidad

2. Inspecciones - FORMAL

Definición: Tiene **proceso formal** y **conjunto de roles**

Requisito: Necesaria **utilización de checklist**

- Ayuda a memoria para saber qué cosas controlar

Características:

- Se toman **métricas**
- Reporte de revisión al final de inspección
- Analiza defectos encontrados

Garantía: Actividad que **garantiza calidad software**

- Éxito depende de planificación

Objetivos:

1. Descubrir **errores**
2. Verificar que software alcanza **requerimientos**
3. Garantizar que software fue construido de acuerdo con **estándares**
4. Conseguir software desarrollado de manera **uniforme**
5. Hacer que proyectos sean más **maneables**

Características de Procesos:

- **Time boxing** (bloques de tiempo fijo)

- Exigen **alto esfuerzo intelectual**

Roles Participantes (obligatorios):

1. **Autor:** Creador/encargado de mantener producto a inspeccionar
 - Inicia proceso seleccionando moderador
 - Junto a moderador eligen resto de roles
 - Entrega producto a ser inspeccionado al moderador
2. **Moderador:** Planifica y lidera revisión
 - Trabaja con autor para elegir demás roles
 - Entrega producto a inspector 2 días antes reunión
 - Coordina reunión (evita conductas inapropiadas)
 - Realiza seguimiento defectos encontrados
3. **Anotador:** Registra hallazgos inspección
 - Usualmente termina confeccionando reporte revisión
4. **Lector:** Lee producto a ser inspeccionado
 - Necesario para que participantes no se dispersen
5. **Inspector:** Examina producto antes reunión para encontrar defectos
 - Registra tiempos de preparación
 - **TODOS pueden ser inspectores. TODOS pueden inspeccionar.**

Nota: Ciertos roles pueden ser asumidos por la misma persona

Etapas del Proceso de Inspección:

1. **Planificación**
 - Moderador (a pedido autor) planifica inspección
 - Define: lugar, tiempo duración, roles
 - **Duración MAX 2 horas** (alto esfuerzo intelectual)
2. **Visión General (OPCIONAL)**
 - Autor realiza descripción general del producto a inspeccionar
3. **Preparación**

- Preparación de cada rol para reunión
- Cada rol adquiere copia de producto trabajo
- Deben **leer y analizar** para encontrar potenciales defectos
- Permite que reunión inspección sea más **productiva**

4. Reunión de Inspección

- Equipo realiza **análisis** para recolectar potenciales defectos previos
- Descartar **falsos positivos**
- Lector lee producto trabajo
- Inspectores comparten defectos encontrados (registrados por anotador)
- Reunión finaliza con **conclusión**: acepta o NO el producto
- Realiza **informe** detallando: qué se revisó, por quién, qué se descubrió, conclusiones

5. Corrección

- Finalizada reunión, autor realiza **correcciones** de defectos encontrados

6. Seguimiento

- Dependiendo **gravedad** puede existir proceso de re-inspección
- Defectos graves = re-inspección
- Defectos simples = autor solo los corrige
- Otros defectos sin re-inspección = autor se reúne con moderador para tratar correcciones

Resumen: Revisiones Técnicas SON vs NO SON

SON:

- Forma más barata y efectiva de encontrar fallas
- Forma de proveer **métricas al proyecto**
- Buena forma de proveer **conocimiento cruzado**
- Buena forma de promover **trabajo en grupo**
- Método probado para mejorar **calidad del producto**

NO SON:

- Utilizadas para **encontrar soluciones a fallas**
- Usadas para **obtener aprobación de producto trabajo**
- Usadas para **evaluar desempeño de personas**

 Testing

 Testing en agile

 Filosofía Lean

 Métricas en enfoques de Software

 Nexus

 TDD: Test Driven Development

 Aseguramiento de la Calidad